

Reproducible Pipeline for Sharp-Wave Ripple Detection in LFP Recordings

This project implements an end-to-end computational pipeline for detecting sharp-wave ripple events from local field potential recordings. The code is organized around a reproducible train/test workflow that loads neural recordings, extracts time-frequency features, learns a ripple-sensitive scoring function, exports model-ready datasets, and evaluates predictions at both the window and event level.

The main pipeline is defined in `_run_pipeline.py`, with supporting signal-processing and modeling utilities in `_helpers.py`. The command-line scripts support experiments such as `two-class` SWR versus NonSWR classification and `three-class` HighSWR, LowSWR, and NonSWR classification.

The pipeline begins by loading session-level LFP data and ground-truth ripple annotations. Training sessions use supervised channel selection, choosing the channel with the strongest ripple-band envelope inside known ground-truth events. Test sessions use an unsupervised ripple-band envelope criterion, making the method usable when labels are unavailable. Each selected channel is transformed with an STFT, restricted to frequencies up to 600 Hz, then normalized through row-wise median flattening and power compression. This preprocessing reduces broadband $1/f$ structure and emphasizes ripple-relevant spectral patterns.

A central strength of the code is its joint matrix factorization stage. The function `fit_joint_nmf_and_score` fits a joint NMF model across train and test recordings, producing shared spectral templates and time activations. From these components, the pipeline reconstructs time-frequency structure and learns a frequency weighting function, $g(f)$, using a linear SVM trained only on labeled training frames. The resulting ripple score, $S(t) = g^T \hat{X}$, converts each recording into a continuous ripple-sensitive trace that highlights candidate ripple activity.

Training data are then built by combining ground-truth SWR windows with mined NonSWR windows. The `two-class` pipeline exports SWR and NonSWR examples, while the `three-class` pipeline further separates ripple events into HighSWR and LowSWR using local contrast scores and Gaussian mixture thresholding. These exports are written as per-session and combined CSV files. They include metadata such as session ID, channel, sample bounds, event duration, NMF activation summaries, ripple-band ratios, and raw LFP windows. This makes the output suitable for downstream models such as Informer, MLP, or classical feature-based classifiers.

The test pipeline generates candidate windows either from detected peaks in the learned ripple score or from all H/STFT bins. These candidates are exported without using labels, then later matched against ground truth for evaluation. Evaluation is handled in `_Evaluation.py`, which reports both window-level and event-level performance. This distinction is important: window

metrics measure classification quality on candidate segments, while event metrics measure whether true biological ripple events are recovered with acceptable timing.

One completed Informer-style evaluation trained on **Amigo2 + Som2** and tested on **Dlx1 + Thy7** achieved a best window F1 of approximately **0.528**, with event-level precision of **0.643**, event-level recall of **0.611**, and event-level F1 of **0.626**. The median timing error was approximately **-1.73 ms**, indicating that detected events were temporally close to ground-truth ripple centers.

In the final version of the pipeline, the model is not trained only on peak-detected candidate events. Instead, the whole NMF-derived time representation is fed into the train and test stages. Each NMF/STFT time bin, or H bin, is converted into a fixed-width LFP window and exported as a model input. This means the classifier sees the full temporal structure produced by the NMF decomposition rather than a pre-filtered set of peaks.

For training, every H bin is labeled according to its relationship with the ground-truth ripple annotations. Bins whose centers fall inside or near annotated SWR events are treated as positive examples, while bins outside those regions are treated as NonSWR examples. For testing, the same full-bin strategy is used: every NMF time bin is exported as a candidate window, and predictions are later compared against ground truth during evaluation. This avoids biasing the model with a peak detector before classification.

Using this final strategy, the model was trained on **Amigo2 + Som2** and evaluated on **Dlx1 + Thy7**. With the SVM model at a threshold of **0.35**, the pipeline achieved an event-level F1 score of **0.701**, with precision of **0.743** and recall of **0.663**. XGBoost also performed strongly, reaching event-level F1 of **0.672**, precision of **0.722**, and recall of **0.628**. Other neural architectures are still in progress. In particular, additional tuning and evaluation are ongoing for deep models such as LSTM, **1D-CNN**, **2D-CNN**, and Transformer/Informer-based variants.

To test whether the proposed pipeline adds value beyond the rippleAI workflow, it was compared against a reproduced rippleAI baseline trained directly on raw LFP. In the reproduction study, rippleAI was trained on the original split, **Amigo2 + Som2**, and evaluated on **Dlx1 + Thy7**. The best reproduced raw-LFP rippleAI model was the LSTM, with an F1 score of **0.656**, while the mean F1 across the **five** rippleAI architectures was **0.605**. The reproduced raw-LFP rippleAI baseline achieved F1 scores of **0.600** with SVM and **0.591** with XGBoost.

Overall, the proposed pipeline outperformed the strongest reproduced rippleAI raw-LFP result even when using a simpler classifier. The reproduced rippleAI baseline achieved its best performance with LSTM, reaching F1 **0.656** on the **Amigo2 + Som2** to **Dlx1 + Thy7** split. In contrast, the proposed NMF-fed pipeline reached event-level F1 **0.701** using SVM. This suggests that the improvement is not only due to model complexity, but also comes from the quality of the NMF-based representation itself.